

ICE_PLANT Project Documentation

Elliot Shabram

Primary Contact: elliottshabram@gmail.com

Institutional Contact: egshabram@ucdavis.edu

April 1, 2026

This project is a USB dongle for collecting data from a Corometrics 250cx and viewing it live. The work is split across two repositories. `ICE_PLANT` contains the code that runs on the Raspberry Pi Zero 2 W, and `ICE_PLANT_VIEWER` is the desktop application used to connect to it and display the data. I also used Philips Series 50 material as a guide to the packet format and payload layout, but the monitor in use here is the Corometrics unit.

Repositories

Source code for both projects is hosted on GitHub here:

- `ICE_PLANT`: [ICE_PLANT GitHub Repository](#)
- `ICE_PLANT_VIEWER`: [ICE_PLANT_VIEWER GitHub Repository](#)

1 Embedded Device (`ICE_PLANT`) and Wiring

`ICE_PLANT` is the repository for the embedded code in the Ice Plant project. All the files needed for the custom image, including setup scripts, service setup, and the Ice Plant application are housed here.

The main runtime file is `ice_plant.py`, which can be manually run, but is set up as a Linux service that starts at boot. The application talks to the Corometrics device via the RS-232 port on the MAX3232 over UART/TTL, and communicates over the TTL serial for starting and reading data from the Corometrics machine. It can be run with or without a network connection, and is specifically targeted to work with enterprise networks; hence the choice of a higher-level operating system running on the RPi than a typical low-power dongle device. Because of the higher-level operating system, the device has some added network capabilities that it would otherwise not have, such as the ability to SSH in to the system and access it fully behind a VPN.

Signal Path

The hardware chain is as follows:

Corometrics 250cx -> custom RJ-11/DB9 cable -> MAX3232 -> Pi Zero 2 W UART/TTL Serial

Here is the mapping of the custom cable from the Corometrics machine to the DB9 interface used by the MAX3232 board for reference:

Corometrics RJ-11	DB9 Pin	Use
Pin 5 (TXD)	2 (RXD)	Monitor transmit to host receive
Pin 2 (RXD)	3 (TXD)	Host transmit to monitor receive
Pin 3 or 4 (GND)	5 (GND)	Signal ground reference
Pin 1 (RTS, optional)	7 (RTS)	Optional hardware flow control
Pin 6 (CTS, optional)	8 (CTS)	Optional hardware flow control

From the TTL side, MAX3232 TX goes to Pi GPIO15 (RXD0), MAX3232 RX goes to Pi GPIO14 (TXD0), where the ground is shared by both. The MAX3232 board is only used for voltage-domain translation between RS-232 and 3.3V UART, and was chosen primarily because of its standard use in serial communication. Another style connector could have been chosen as well, but this configuration was chosen because it felt like the most straightforward way to get data in through serial. Also, the design choice of going through TTL instead of something like USB was intentional in order to utilize the USB male and female connectors for other purposes such as powering the device or connecting over serial gadget. In the future, if a newer machine were to provide a data capable USB port, the Ice Plant device could easily be adapted to that for proper dongle style function, but for now, it is handy to have the USB male free.

Embedded Runtime Behavior

The main file for ICE_PLANT is `ice_plant.py`. It runs as a Linux service so it starts up at boot. When started, it opens `/dev/serial0` and enters a polling phase by sending a CTG request block. When data first appears, it sends the command that enables periodic auto-send behavior and then stays in a continuous data reading mode. Even when the sensors are recording only zero values, the device still logs them so the time-domain data stays consistent, and the viewer application continues to tick forward.

In the continuous mode, bytes are read into a rolling buffer. The parser looks for the frame boundaries, pulls out complete frames, checks the CRC-16, and then removes the DLE escaping characters. Invalid frames are discarded and counted for status purposes. Valid payloads of any block type are written to CSV.

Operationally, this creates a simple state machine with these states: startup, poll-until-data, switch-to-stream, then stream-validate-log forever. If bytes keep arriving but no valid frames show up for several seconds, the logger clears the input buffer and runs the poll/go handshake again. That recovery step turned out to be necessary because in some situations the machine stops transmitting when it has nothing to send.

Protocol Handling at the Embedded Layer

Framing handled by the logger is:

DLE STX <escaped payload> DLE ETX <CRC16>

The CRC is computed on the block up to/including DLE ETX, which is then compared to the trailing two-byte CRC field for validation. If the packet is found to be valid, the payload bytes are then logged.

After framing and CRC validation, the logger writes payload bytes to CSV as received. Decoding of the raw monitor data in the payload is done by the viewer application, so the embedded side

can concentrate on recording rather than more intensive parsing. The first payload byte is also written separately as the CSV `block_type` field so different payload types can be grouped without re-parsing the full payload. Other than the extracted type byte, the payload is left untouched.

Data Contract and File Layout

The CSV format is like so:

```
timestamp,block_type,payload_len,payload_hex
```

Example file naming:

```
ctg_frames_YYYYMMDD_HH00.csv
```

The CSV format is standard with the header created at the top describing the columns, and each entry is appended as the rows. Files are created each hour, where at least one valid payload is accepted during that hour. We enforce both a maximum file count, as well as a check for free space. If either limit is exceeded, the oldest files will be deleted first (FIFO). This scheme keeps long-running behavior predictable on limited SD storage while still allowing for scaling with larger cards. In practice, the resulting hourly files are highly uniform in size.

Any CRC-valid payload is recorded, including standard CTG blocks (C) and maternal oxygen saturation blocks (S). Other Philips-style packets which are defined in the Philips programmer's guide were not verified, and it's not clear whether they are even used by the Corometrics 250cx.

Storage Use Evaluation

A long test was run from 2026-03-12 18:00 through 2026-03-31 20:00 (as of finalizing this document) which produced 459 consecutive hourly CSV files with no missing hours. For this test, the device was plugged in to the first RJ11 port on the rear of the Corometrics machine and left to run for several weeks. As of now, the test has been running for a little over 19 days without issue, with all files close to 466 KiB, indicating that the data is consistent. File sizes were tightly clustered, ranging from 465,511 bytes to 466,345 bytes, with a mean of 466,099.20 bytes and a standard deviation of 93.85 bytes. The full observed spread of sizes was only 834 bytes, or about 0.18% of the mean hourly file size, which shows that data will be somewhat predictable on storage space.

A healthy deployment should ideally produce a continuous chain of hourly files with sizes close to the baseline we see here. Missing hours would indicate that no valid payloads were accepted during those intervals, which would further indicate that the machine was off, or the device was unplugged. Substantially undersized files would also suggest some sort of downtime. Since we are refreshing the poll often, there should be data, even when all returns are zeroes.

Using the observed mean hourly size, projected storage usage will be approximately 0.31 GiB for 30 days of capturing continuously, and 3.80 GiB for one year of continuous capture. These numbers are quite low, and make ICE_PLANT an excellent option for very long term data logging.

At this observed rate, the built-in 64000-file retention cap in `ice_plant.py` corresponds to roughly 2667 days of hourly files (about 7.3 years), or about 27.8 GiB of CSV data, before old files are rotated out by file-count limit. Independently of the file count, the logger also deletes the oldest files if free disk space drops below 250 MiB.

Corometrics 250cx Packet Types

The payload type that is typically seen from the Corometrics machine is the Philips-style CTG data block **C**, which is returned when polling. When attaching the maternal pulse oximetry sensor, the monitor also emits an **S** block for maternal oxygen saturation, which we report in the viewer application. Note that this is distinct from the protocol's **FSpO2** byte inside the **C** block, which refers to fetal oxygen saturation, and is less often reported. Testing was somewhat limited in this area because of the availability of sensors. All sensors that were available were plugged in and tested in various scenarios and conditions.

S payload layout is:

Byte 0: 'S'

Byte 1: maternal SpO2 at 0.5 % resolution

Bytes 2-3: maternal pulse rate at 0.25 bpm resolution, MSB first

Here are a few examples:

53 be 0154 -> 95.0 % SpO2, 85.0 bpm

53 bc 0138 -> 94.0 % SpO2, 78.0 bpm

Unlike the per second CTG polling data that is dominated by the **C** blocks, **S** blocks would appear asynchronously. It was hard to determine a pattern with these blocks, as the documentation is very limited on this, and there was no consistent interval that could be seen. The logger records them like any other packet, and the viewer application reports the last known maternal SpO2 value until a new **S** block arrives or becomes stale (somewhat subjectively chosen staleness).

Embedded Deployment

Setup scripts are housed in the **firmware/** directory that configure the fresh Linux image. The script **setup.sh** moves assets into runtime locations, enables system services, and configures serial gadget access. For creating a new Linux image, see the ICE_PLANT repository's README.md.

Network Setup

One of the main hurdles to designing IoT devices is enterprise networks because of their heavy authentication and authorization schemes, for which higher-level systems and often GUIs are needed to gain access. Each enterprise network varies in its implementation and level of security. For this project, the UC Davis **eduroam** enterprise network was used in the Ice Plant device. This required configuring a special WPA supplicant configuration via the **wpa_supplicant-wlan0.conf**, and the inclusion of a special certificate (both included in the **firmware/** directory). The certificate for the **eduroam** was obtained online, and had to be manually extracted to be used with our configuration scheme.

The next challenge faced with networking the Ice Plant device was access and communication from behind the Network Address Translation (NAT), which hides individual devices behind a single IP with the assignment of local IPs. For this, the Tailscale VPN was chosen as the solution because of its free and simple operation, allowing the user to maintain a virtual network of personal devices for managing access. Tailscale is installed via the **setup.sh** script upon creation of a new image, and sign-on is done on first boot, described in the ICE_PLANT repository's README.md.

Simulation for Development

During development it was necessary to have a way of simulating data as closely to the Corometrics as possible so that development could continue without being physically connected to the monitor. For this, the `simulate_data.py` was created, which generates protocol-compatible synthetic payloads into `ctg_frames_sim_*.csv`.

The simulator purposely mirrors the live CSV format so the viewer application can read it in the same way. However, the viewer application will discriminate between the files denoted with the term "sim", warning you that you are watching simulated data, and ignoring them when downloading data files. It also simulates some simple contractions and changes in BPM, as well as some mixed in randomness to give it more realistic ranges and behavior.

Embedded Failure Behavior

If the serial device throws an exception, the process closes the port, waits briefly, and tries again to open the interface. CRC failures won't crash the process. Frames are simply dropped and logging continues. If no new data arrives, the process will remain alive and keep reading. If bytes arrive continuously but don't produce a valid frame, the logger will treat that as a recoverable fault and reinitiate the stream.

The logger is generally designed to fail soft at frame level and continue running even when there is no data arriving. There is always more that can be done for testing and safety, but I spent quite a few hours testing and am confident in the state of both the embedded device and the viewer application. However, bugs are inevitable from a team of one, such as myself, so safety and hardness of operation should be treated as one of the areas in need of future work.

2 ICE_PLANT_VIEWER Application

The viewer is the analysis/visualization side of the project, and was an additional feature that I decided to add given the ample time. It communicates with the embedded application running on the RPi over SSH when on the network, which was chosen for its simplicity. Future work could see a dedicated TLS server for this project, but this would also require additional infrastructure. When connected via the serial USB, the viewer application communicates with the device over the serial gadget directly without the need for SSH by logging in and tailing the CSV files with shell commands.

To connect to the ICE_PLANT device, you must be on the same network. That means that for remote monitoring, some kind of VPN will need to be used. Currently, the setup script installs Tailscale for this, with instructions on how to add the device to your personal network in the ICE_PLANT README.md. Once on the same network, enter the hostname or IP as you would an SSH command. Ex:

```
ssh pi@10.0.0.1
```

The application will save your previously connected remote device and auto-reconnect for your convenience. Note that password authentication is currently enabled, but can be easily turned off once public keys from the user's devices have been added to the RPi's SSH configuration.

Remote Stream Pipeline

The viewer starts by connecting to the RPi over SSH, locating the most recently created CSV in `/home/pi/ICE_PLANT/data`, and tailing the appended lines. This is done in a worker path so the UI can remain responsive.

When hourly rotation of data files occurs on the embedded side, the viewer detects the new file and switches to that file to maintain the most current data in the visualization. There is also an option to prefill all data contained within the latest file, which can be useful if the user needs to see a longer timeline.

Parsing and Decode

For the live files, each CSV row has the timestamp, the block type, the payload length, and the payload hex. The payload is broken down into the various components, which is then plotted using its timestamped value along the x-axis. The first tab contains the plotted heart-rates for maternal and two fetal signals, as well as SpO2 for the fetal signal. The maternal SpO2 is reported above with some other values. The second tab contains the TOCO uterine signal.

Since the heart-rates are stored in quarter-bpm units, the viewer application divides them by 4 before plotting them. The TOCO values are also stored as half-values, so we divide them by 2 before plotting.

Visualization and UI Loop

The UI is built using PySide6 for its GUI components, as well as Matplotlib for the data plotting. The viewer has two main tabs for viewing separate heart-rate trends and uterine contraction data. The application is capable of connecting to the device over USB gadget serial, which allows for live monitoring without a network. This feature could be utilized to implement any custom algorithms and data processing that researchers might want to see in the future. Also, the application allows for a serial terminal to be opened within the application, simplifying the process of accessing the system via serial.

Viewer Failure Behavior

SSH errors, remote file read errors, and malformed rows are handled without taking down the full application when possible. If connection is lost, capture on the Pi is unaffected. Reconnection resumes from the current log state.

This separation prevents remote UI issues from corrupting acquisition.

Packaging and Distribution

The viewer repository includes dependency pins and PyInstaller build scripts for desktop distribution, including macOS packaging scripts. I'll also provide a prebuilt installer for macOS and Windows, which is published in the repository's GitHub Releases section. The current (first) version of the software is v0.1.0, which will increment automatically when a GitHub build is run. For instructions on running from source or building and deploying the application, see the `README.md`.

3 Downloading Data

The Viewer application functions as the avenue for downloading data from the ICE_PLANT embedded device through the top menu options. Simply click **File -> Download Data Files...** and select the file location for download. This can be done either remotely or when connected through the USB gadget serial connection (USB male). Downloading via the USB gadget is a highly useful feature that allows the monitor to function entirely off-network if need be. Since this project is designed with enterprise networks in mind, it was necessary to have this kind of fallback as enterprise networks vary greatly and are notoriously difficult to set up with IoT devices. If an organization suddenly switches IT departments or ISP, ICE_PLANT will still collect data, and the viewer application can still download it. Note that the files labeled with "sim" are ignored when downloading data.

The second mode of remotely retrieving data is via the `rsync` command. You can use this:

```
rsync -avz pi@<pi-hostname-or-ip>:~/ICE_PLANT/data/ ./
```

4 Future Work

Future work should focus on reliability and maintainability of the existing pipeline, but could also benefit from added features where desired.

On the embedded side, one remaining gap is serial recovery. The logger does attempt recovery after read failures, but if the serial port cannot be reopened immediately, that path should keep retrying instead of failing out. Also, much of the configuration is hard-coded into the application, and would benefit from a more standard scheme. Lastly, the remote path currently relies mainly on SSH, which is simple and secure, but not ideal for an embedded application like this. One solid standard solution to this would be a TLS server that we control. This would require additional server infrastructure, and would complicate the setup a little more.

For the viewer application, the most useful improvements are more subjective, like adding features, but some things remain to be improved. For one, the reconnect behavior could be improved upon. Additionally, for validation, the project should add some standard regression tests so that the parser and decoder changes can be checked quickly before deployment.

Finally, I think that the viewer application would be a great place to implement any analysis algorithms that are used with this data. The application is simply written in Python, and would be easy to integrate. The user would also benefit from a real-time view of the final analysis.

5 Conclusion

The Ice Plant project successfully achieved the original goals of creating a USB dongle for remote retrieval of data from the Corometrics 250cx connected over an enterprise network. The project later extended to include the viewer application for better usability and ease of data management. Much testing was done throughout the lifecycle of the project, particularly in the last two months, and showed that Ice Plant is robust and field-ready.